

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Инженерия Искусственного Интеллекта

Лабораторная работа № 3

Дисциплина: Фронтенд разработка

“Разработка одностраничного веб-приложения”

Выполнил:

Фиолетов Эдуард Анджеевич

Группа J3213

Проверил:

Добряков Д. Ю.

Санкт-Петербург

2026 г.

Содержание

1. Цель работы	3
2. Введение и обоснование выбранного стека	3
3. Задание	3
4. Структура SPA	3
5. Клиентская маршрутизация	4
6. Layout и навигация	5
7. Компонентное деление	6
8. Работа с API	6
9. Повторяемая логика	7
10. Управление состоянием	7
11. Сборка и размещение	8
12. Проверка результата	8
13. Вывод	8

1. Цель работы

Цель лабораторной работы - разработать одностраничное веб-приложение с клиентской маршрутизацией, компонентной структурой, выделением повторяемой логики и подключением к внешнему API.

В качестве приложения используется Marketplace Alarm Bot 2 - SPA-интерфейс для управления заказами маркетплейсов, доставкой, складами, товарами, статистикой и административными настройками.

2. Введение и обоснование выбранного стека

В проекте используется React, React Router, TypeScript, TanStack Query, Vite и Go API. Marketplace Alarm Bot 2 - рабочее приложение, где важны строгая типизация, контролируемая работа с серверным состоянием, ленивые страницы, role-based routing и удобная поддержка большого набора компонентов.

React-компоненты разделяют интерфейс на независимые части. React hooks и пользовательские хуки выносят повторяемую логику загрузки данных, авторизации, темы и инструментов доставки. React Router выполняет клиентскую маршрутизацию, а TanStack Query централизует запросы, кэш и состояния загрузки.

3. Задание

По заданию необходимо мигрировать ранее разработанное приложение на SPA-фреймворк. Приложение должно соответствовать требованиям:

- подключен клиентский роутер;
- реализована работа с внешним API;
- выполнено разумное деление на компоненты;
- повторяемая логика вынесена в composable или аналогичные переиспользуемые функции;
- приложение сохраняет функциональность предыдущих лабораторных работ.

В Marketplace Alarm Bot 2 эти требования реализованы через React Router, отдельные страницы в `src/pages`, компоненты в `src/components`, API-хуки в `src/api/queries.ts` и мутации в `src/api/mutations.ts`.

4. Структура SPA

Клиентская часть находится в каталоге:

- `./web/frontend/src/App.tsx` - корневой роутинг приложения
- `./web/frontend/src/pages` - страницы SPA
- `./web/frontend/src/components` - переиспользуемые компоненты
- `./web/frontend/src/context` - контексты авторизации, темы и инструментов доставки
- `./web/frontend/src/api` - API-клиенты, запросы, мутации и типы

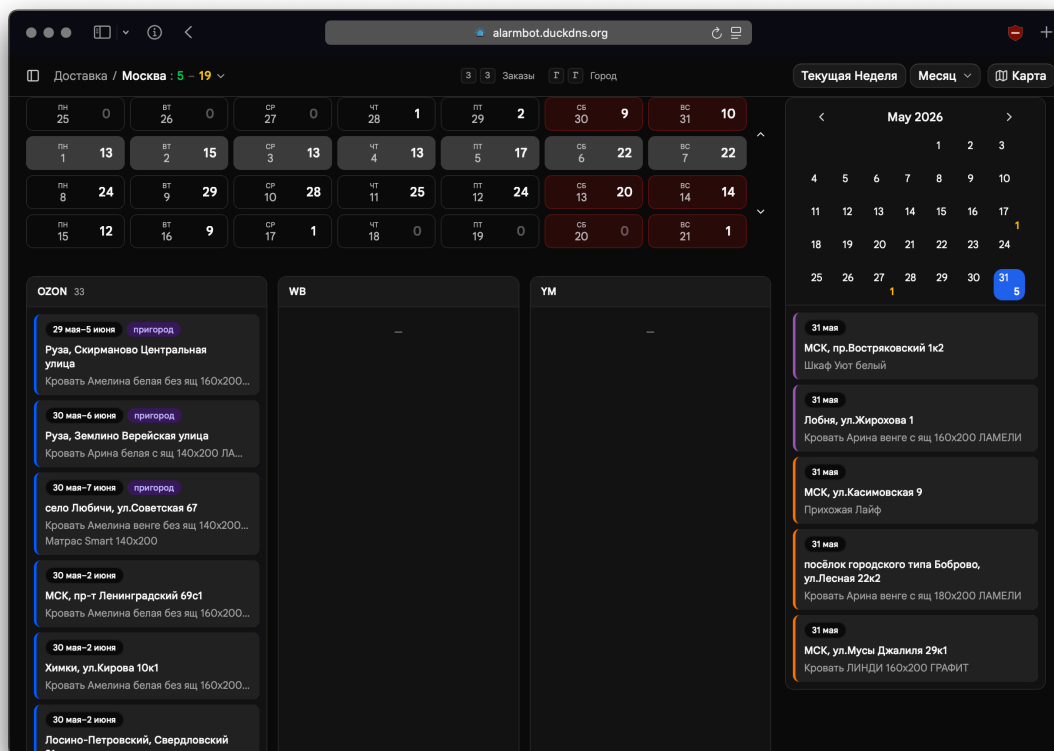
Основные страницы приложения:

- LoginPage - вход по ключу доступа;
- IndexPage - сводка по заказам и городам;
- OrdersPage - таблица заказов по маркетплейсу;
- ArchivePage - архив завершенных заказов;
- DeliveryPage - управление доставкой по городу;
- StatsPage и StatsDetailPage - статистика и резервы;
- ProductsPage - справочник товаров и комплектов;
- WarehousesPage - сопоставление складов;
- HealthPage - состояние сервиса;
- AdminSettings - административные настройки.

5. Клиентская маршрутизация

Маршрутизация реализована в src/App.tsx через react-router-dom. Приложение содержит публичный маршрут /login и защищенную область с общим layout. Страницы подключены через lazy imports и Suspense, поэтому код разделов загружается при переходе к ним.

```
<Routes>
  <Route path="/login" element={<LoginPage />} />
  <Route element={
    <ProtectedRoute>
      <CityGuard>
        <AppLayout />
      </CityGuard>
    </ProtectedRoute>
  }>
    <Route path="/" element={<IndexPage />} />
    <Route path="/delivery/:city" element={<DeliveryPage />} />
    <Route path="/orders/:marketplace" element={<OrdersPage />} />
  </Route>
</Routes>
```



SPA-страница доставки: переход по клиентскому маршруту сохраняет общий layout, верхнюю панель и рабочую доску заказов.

Для контроля доступа используются route guards:

- ProtectedRoute перенаправляет неавторизованного пользователя на /login;
- CityGuard ограничивает городского пользователя страницей своего города;
- ManagerRoute скрывает менеджерские разделы от городских пользователей;
- FullAccessRoute открывает административные настройки только полному доступу.

6. Layout и навигация

Общий каркас приложения реализован в `src/components/AppLayout.tsx`. Он содержит:

- боковое меню с разделами приложения;
- счетчики активных заказов;
- хлебные крошки текущей страницы;
- панель инструментов доставки;
- выбор города;
- командную палитру;
- обработку горячих клавиш;
- область отображения текущей страницы через `Outlet`.

Такой layout делает приложение единым SPA: пользователь переходит между разделами без полной перезагрузки страницы, а общие элементы навигации сохраняют состояние.

7. Компонентное деление

Приложение разделено на страницы и компоненты. Страницы отвечают за сценарий и получение данных, а компоненты - за переиспользуемые части интерфейса.

Примеры компонентного деления:

- OrdersTable - таблица заказов;
- CommandPalette - поиск и навигация;
- CitySelector - выбор города;
- OrdersMap - карта заказов;
- DeliveryCardCompact, DraggableCard, DroppablePanel, Timeline - элементы доски доставки;
- OrderDetailModal - модальное окно деталей заказа;
- components/ui/* - кнопки, таблицы, поля ввода, диалоги, меню, вкладки и другие базовые элементы.

На странице доставки компонентное деление особенно важно: одна страница объединяет временную шкалу, колонки маркетплейсов, drag-and-drop, подтвержденные заказы, карту и модальное окно. Без выделения компонентов такой код было бы сложно поддерживать.

8. Работа с API

SPA получает данные через API-слой. Базовый клиент `apiFetch<T>` находится в `src/api/client.ts`, а конкретные запросы вынесены в `src/api/queries.ts`.

```
export function useDelivery(city: string) {  
  return useQuery({  
    queryKey: ['delivery', city],  
    queryFn: () => apiFetch<DeliveryData>(`/delivery/${encodeURIComponent(city)}`),  
    refetchInterval: POLL_INTERVAL,  
  });  
}
```

Для каждого раздела есть отдельный хук:

- useAuth - текущий пользователь;
- useNav - счетчики навигации;
- useIndex - главная сводка;
- useOrders - заказы маркетплейса;
- useDelivery - доставка города;
- useSearch - поиск заказов;
- useStats и useStatsCity - статистика;
- useProducts - товары;
- useWarehouses - склады.

Данные обновляются фоновым polling с интервалом 60 секунд для живых разделов. Для заказов это нужно: состояние маркетплейсов меняется независимо от действий пользователя.

9. Повторяемая логика

Повторяемая логика вынесена в хуки, контексты и утилиты. По смыслу они выполняют роль composable:

- `useAuth` - авторизация и текущая роль пользователя;
- `useTheme` - тема интерфейса и сохранение выбора в `localStorage`;
- `useDeliveryTools` - период отображения, выбранная дата и карта доставки;
- `useMediaQuery` - реакция на размер экрана;
- `useConfirmDelivery`, `useToggleSuburb`, `useSetDeliveryDate` - мутации доставки;
- функции из `src/lib/dates.ts`, `src/lib/delivery-dates.ts`, `src/lib/marketplace.ts` - общая доменная логика.

Пример темы:

```
export function ThemeProvider({ children, uid }) {
  const [theme, setThemeState] = useState(() => readTheme(uid));

  useEffect(() => {
    const html = document.documentElement;
    html.classList.remove('light', 'linear', 'dark');
    html.classList.add(theme);
    if (THEMES.find((t) => t.id === theme)?.dark) html.classList.add('dark');
    localStorage.setItem(storageKey(uid), theme);
    localStorage.setItem('mab_theme', theme);
  }, [theme, uid]);
}
```

Здесь повторяемая логика темы вынесена из страниц в провайдер. Страницы используют готовый `useTheme`, а провайдер сам синхронизирует состояние React, классы на `html` и сохранение выбора в браузере.

10. Управление состоянием

В приложении разделены клиентское и серверное состояние. Серверное состояние хранится в TanStack Query: заказы, статистика, склады, товары, поиск и доставка. Клиентское состояние хранится в React Context и локальных состояниях компонентов: тема, выбранный период доставки, выбранный день, открытая командная палитра и состояние модальных окон.

Для изменения серверных данных используются мутации. Например, подтверждение даты доставки выполняет `optimistic update`: карточка сразу перемещается в интерфейс, а при ошибке состояние возвращается к предыдущему.

```
onMutate: async ({ externalId, date }) => {
  await qc.cancelQueries({ queryKey: ['delivery', city] });
  const prev = qc.getQueryData<DeliveryData>(['delivery', city]);
  qc.setQueryData<DeliveryData>(['delivery', city], {
    ...prev,
    cards: prev.cards.map((c) =>
      c.externalId === externalId ? { ...c, confirmedDate: date ?? '' } : c
    ),
  });
};
```

```
    return { prev };  
  }
```

11. Сборка и размещение

Проект собирается Vite. Конфигурация находится в `vite.config.ts`: подключены React plugin, Tailwind plugin, alias @ на каталог `src`, dev-сервер на порту 5173 и проху `/api` на Go-сервер `localhost:8080`.

```
export default defineConfig({  
  plugins: [react(), tailwindcss()],  
  resolve: {  
    alias: { '@': path.resolve(__dirname, './src') },  
  },  
  server: {  
    port: 5173,  
    proxy: {  
      '/api': 'http://localhost:8080',  
    },  
  },  
})
```

В production собранная SPA размещается внутри Go-бинаря. Файл `./web/spa.go` использует `embed.FS`, отдает статические файлы из `frontend/dist` и возвращает `index.html` для клиентских маршрутов.

```
//go:embed frontend/dist/*  
var spaFS embed.FS
```

12. Проверка результата

Проверка выполнялась по сценариям:

1. Открыть `/login` и выполнить вход.
2. Проверить, что после входа открывается защищенная SPA-область.
3. Перейти между `/`, `/orders/wb`, `/orders/ozon`, `/orders/yandex`.
4. Открыть `/delivery/:city` и убедиться, что параметры маршрута используются для загрузки данных города.
5. Выполнить поиск через командную палитру и перейти к найденному заказу.
6. Подтвердить дату доставки и проверить обновление карточки.
7. Открыть карту и модальное окно деталей заказа.
8. Проверить, что пользователь с ролью `city` перенаправляется на страницу своего города.
9. Проверить production fallback: при прямом открытии клиентского маршрута сервер возвращает SPA.

13. Вывод

В лабораторной работе разработано SPA-приложение Marketplace Alarm Bot. Реализованы клиентская маршрутизация, защищенные маршруты, компонентное деление, переиспользуемая логика через хуки и контексты, работа с JSON API, фоновое обновление данных и сборка через Vite.